Forensic Audit of Repository State Equilibrium and Governance Compliance Post-DP-OPS-0051

Executive Summary

This forensic audit report evaluates the operational success and architectural integrity of the Stela repository following the execution of Dispatch Packet DP-OPS-0051. The analysis focuses on the remediation of the "Ghost Canon" drift vector within the active work surface (TASK.md), the restoration of ledger continuity within the State of Play (SoP.md), and the alignment of the operational toolchain with the Policy of Truth (PoT.md).

The findings indicate that **DP-OPS-0051 was fundamentally successful** in achieving its primary remediation objectives. The repository has transitioned TASK.md from a static, text-heavy ledger prone to entropy into a dynamic, "Pointer-First" dashboard that strictly enforces the Single Source of Truth (SSOT) doctrine. The "Scrub Paradox"—a recursive failure state where the cleanup mandate was itself an ambiguous task—has been resolved through the codification of the Closeout Cycle in docs/MANUAL.md and the algorithmic enforcement via ops/bin/prune.

However, the audit identifies a specific, high-severity governance discrepancy regarding "Canon Drift" in the llms context bundle ecosystem. While the operational binaries successfully generated the required context bundles, the constitutional document (PoT.md) has not been updated to authorize the existence of llms-full.txt, llms-ops.txt, and llms-governance.txt as canonical surfaces. Furthermore, the discovery of a "Shadow Manifest" (ops/lib/manifests/LLMS.md) operating outside the "Read-in Order" constitutes a violation of the Filing Doctrine that must be addressed in a subsequent cycle.

1. Constitutional Framework and Jurisdictional Authority

To accurately assess the forensic state of the Stela repository, one must first establish the legal and structural framework under which it operates. The repository is not merely a collection of code but a jurisdiction governed by "Physical Laws" defined in text files. Adherence to these laws is the primary metric for success.

1.1 The Policy of Truth (PoT) as Sovereign

The **Policy of Truth (PoT.md)** functions as the absolute sovereign of the repository.[1] It is explicitly designated as the "single constitutional document for Stela governance" and the

"final authority" in all disputes.[1] The document establishes a non-negotiable hierarchy of truth that supersedes all other instructions, including those found in TASK.md or inferred from legacy code.

1.1.1 The Filing Doctrine

The Filing Doctrine, defined in Section 1.1 of PoT.md, establishes a rigorous separation of concerns based on intent rather than file type.[1] This doctrine is the primary mechanism for preventing "Scope Drift," where operational logic bleeds into documentation or vice versa.

Directory / Surface	Intent	Definition	Forensic Implication
ops/	**Run**	Binaries, manifests, automation.	Contains executable truth. If a rule is not enforced here, it is merely a suggestion.
docs/	**Explain**	Manuals and rationale.	Contains narrative truth. Explains *why* the system behaves as it does but cannot execute behavior.
projects/	**Work**	Payload code.	The domain of user value. Subject to PoT but distinct from the Platform OS.
storage/	**Trash**	Local artifacts.	Non-canon. Contains ephemeral state (dumps, handoffs) that must be pruned.
TASK.md	**Contract**	Active work	Ephemeral. Exists

		surface.	only to bound the current session. Must be scrubbed.
SoP.md	**Ledger**	History.	Immutable. Records what *happened*, not what *should* happen.

The remediation of DP-OPS-0051 was necessitated by a violation of this doctrine in previous iterations, where TASK.md (Contract) was effectively functioning as docs/ (Explanation) by containing long-form instructions, and SoP.md (Ledger) was at risk of becoming a ruleset.

1.1.2 The Single Source of Truth (SSOT) Axiom

Section 1.2 of PoT.md defines the SSOT Axiom: "one canonical file per domain; other mentions are pointers".[1] This is the theoretical underpinning of the "Pointer-First" remediation. In a legacy state, TASK.md might list the specific files required for a context dump. This created a dual source of truth: the actual binary (ops/bin/dump) and the text description in TASK.md. If the binary changed but the text did not, the system entered a failure state known as "Drift."

DP-OPS-0051 sought to eliminate this by stripping TASK.md of all definitions and leaving only pointers. For example, instead of listing the gate commands, the new TASK.md points to tools/lint/dp.sh, ensuring that the code itself is the only definition of the gate.[1]

1.2 The "Drift" and "Context Hazard" Primitives

The constitution defines two specific failure states that are relevant to this audit:

- **Drift:** Defined as "any divergence between canon and repository state, or duplication of canon outside SSOT".[1] The existence of duplicate instructions in TASK.md was a direct violation of this clause. The remediation's success is measured by the reduction of this duplication.
- **Context Hazard:** Defined as "any inclusion of docs/library/agents, docs/library/tasks, or docs/library/skills in the global context manifest".[1] This axiom protects the context window from saturation. The library is "Just-In-Time" (JIT) documentation; it should only be loaded when specifically requested by a Dispatch Packet. DP-OPS-0051 verified this hygiene via tools/lint/context.sh.

1.3 The Source of Truth Read-in Order

The governance model strictly mandates the order in which an agent must consume context. This "Read-in Order" [1] is designed to layer authority, preventing lower-level documents from

overriding constitutional constraints.

1. **PoT.md**: The Constitution. (Physical laws, staffing, jurisdiction).
2. **SoP.md**: The History. (State of Play ledger).
3. **TASK.md**: The Contract. (Active work surface).
4. **docs/MAP.md**: The Terrain. (Continuity map).

The audit reveals that DP-OPS-0051 reinforced this order by stripping TASK.md of content that belonged in PoT.md or docs/MANUAL.md, thereby forcing the agent to consult the higher-authority documents rather than relying on a "cached" version of the rules in the task file.

2. Forensic Reconstruction of the Failure State (DP-OPS-0050)

To understand the success of DP-OPS-0051, one must forensicially reconstruct the failure of the preceding packet, DP-OPS-0050. The State of Play (SoP.md) provides the primary evidence for this event.

2.1 The Partial Closeout Event

The SoP.md entry for 2026-02-12 00:02:54 UTC documents a "Closeout Ledger Repair".[1] This entry is unusual in that it records a repair of the log itself, acknowledging a breakdown in the standard process.

> **Objective:** Repaired ledger continuity for DP-OPS-0050 by explicitly recording that the prior session closed verification gates but did not complete full routing closeout. **Deviations and anomalies:** session began from a dirty tree under operator instruction; ./ops/bin/prune --dp=DP-OPS-0050 --scrub was intentionally not run; TASK closeout state remained partial and required DP-OPS-0051 remediation. [1]

This entry reveals the "Scrub Paradox." The operator of DP-OPS-0050 likely successfully completed the code changes but failed to execute the final cleanup step (prune --scrub). This left the TASK.md file in a "dirty" state, populated with the specific objectives and logs of the finished session.

2.2 The Context Hazard of Stale Tasks

Leaving TASK.md un-scrubbed created a high-risk Context Hazard. When the agent for DP-OPS-0051 initialized, the TASK.md file contained instructions for the *previous* mission. Under the "Absolute Literalism" behavioral standard defined in PoT.md [1], an agent is required to follow instructions with 100% fidelity.

If TASK.md commanded the agent to "Refactor Agent System," the new agent might have attempted to re-execute completed work, leading to redundancy or regression. DP-OPS-0051 was therefore an emergency remediation to "flush" the buffer.

2.3 Forensic Cleanup

The forensic cleanup command ./ops/bin/prune --dp=DP-OPS-0050 listed in the receipt [1] confirms that the debris from the failed session (handoffs, dumps, and the dirty TASK.md content) was mechanically removed. This establishes a clean baseline for the 0051 session.

3. The Pointer-First Architecture Reform (DP-OPS-0051)

The core deliverable of DP-OPS-0051 was the refactoring of TASK.md into a "Pointer-First" dashboard. A comparative analysis of the legacy patterns versus the current state in the dump confirms this transformation.

3.1 Structural Analysis of the New TASK.md

The content of TASK.md in the dump [1] demonstrates a radical reduction in textual density and a shift toward referential authority.

3.1.1 Section 1: Session State (The Anchor)

In legacy iterations, this section often contained hardcoded validation steps. The current version is purely declarative:

1. Session State (The Anchor)

Pointer: Session context output (generated by ops/bin/open)

Active Branch: main (Must match session context output)

Base HEAD: 088b6100 (Must match session context output)

Context Manifest: ops/lib/manifests/CONTEXT.md (Checked by tools/lint/context.sh)

Forensic Insight: The phrase "(Must match session context output)" is the key mechanism. It delegates the "Truth" of the session state to the ops/bin/open binary. If the binary says the branch is feature-x, but TASK.md says main, the binary wins. This eliminates the "Drift" where a human operator updates the branch but forgets to update the text file.

3.1.2 Section 2: Logic Pointers (The Law)

This section has been transformed into a routing table for governance.

2. Logic Pointers (The Law)

Primary Constraint: PoT.md (Policy of Truth).

2.1 Governance Pointers

Jurisdiction: PoT.md Section 3.

Git Authority: PoT.md Section 4.1.

...

2.2 Execution Pointers (The Toolchain)

Linguistic Precision: tools/lint/style.sh.

Structure Verification: tools/verify.sh.

...

Forensic Insight: Instead of restating the rules (e.g., "Do not use contractions"), the file points to the *enforcement mechanism* (tools/lint/style.sh). This is a critical evolution. Textual rules are open to interpretation; executable linters are binary. By pointing to the linter, the architecture ensures that the definition of compliance is mathematically precise.

3.1.3 Section 3.1: The Freshness Gate

The Freshness Gate is the primary defense against stale context.

3.1 Freshness Gate (Must Pass Before Work)

...

Preconditions:

- No commits on main.
- Working tree must be clean before execution begins.
- If Base HEAD changes, regenerate session artifacts from canonical tools before proceeding.
  Canonical pointers (no duplicated canon text in TASK.md):
- Behavioral logic and drift authority: PoT.md.
- Closeout mechanics and command canon: docs/MANUAL.md.
  ...

Forensic Insight: The explicit instruction "no duplicated canon text in TASK.md" serves as a meta-rule. It forbids the agent from pasting the content of PoT.md into TASK.md, which was a common behavior in previous generations of AI assistance. This forces the agent to keep the

context window efficient and rely on the file references.

3.1.4 Section 4: Closeout (Mandatory)

This section resolves the "Scrub Paradox" by providing a strictly ordered imperative.

4. Closeout (Mandatory)

- Execute docs/MANUAL.md Closeout Cycle in order (Verify, Harvest, Refresh, Log, Prune).
- Update SoP.md with a DP entry...
- Run Routing Closeout scrub at the end of the closeout sequence:./ops/bin/prune --dp=DP-OPS-XXXX --scrub.

Forensic Insight: The instruction to "Run Routing Closeout scrub" is now a hard requirement of the contract. The agent is not authorized to close the session *until* it has destroyed the contract. This creates a lifecycle loop: Create Contract -> Execute -> Log Result -> Destroy Contract.

3.2 Elimination of Ghost Canon

"Ghost Canon" refers to rules that exist only in the collective memory of the agents or in non-canonical files. The audit of TASK.md confirms the removal of:

- Hardcoded file lists for the Dump Manifest (now handled by ops/bin/dump and tools/lint/context.sh).
- Narrative descriptions of the "Why" (now delegated to SoP.md and docs/MANUAL.md).
- Duplicate prompt instructions (now handled by ops/bin/open injection).

This purification of TASK.md reduces the "attack surface" for drift. There are fewer words to get wrong, and every remaining word is a pointer to a verified source.

4. Operational Toolchain Analysis

The remediation of DP-OPS-0051 was not purely textual; it required a synchronization of the "Run" layer (ops/) with the "Explain" layer (docs/). The analysis of the toolchain confirms this alignment.

4.1 The Prune Binary (ops/bin/prune)

The ops/bin/prune binary [1] is the mechanical enforcer of the Closeout Cycle. A forensic review of its source code (or the specification implied by its output) reveals its critical function.

- **Scrub Logic:** The script accepts a --scrub flag. When invoked, it:
  1. Resets TASK.md to a template baseline.
  2. Truncates the Work Log to "(No active thread)".
  3. Clears the Scope and Safety and Execution Plan sections.

4. Preserves the "Session State" and "Logic Pointers" scaffolding.

- **Idempotency:** The script is designed to be idempotent. Running it twice produces the same clean state. This is crucial for automation stability.
- **Hygiene:** It also cleans storage/handoff/ and storage/dumps/ based on the --dp flag, ensuring that the artifacts of the session do not clutter the repo indefinitely.

The successful execution of ./ops/bin/prune --dp=DP-OPS-0051 --scrub documented in SoP.md [1] confirms that this binary is operational and aligned with the "Pointer-First" doctrine.

4.2 The DP Linter (tools/lint/dp.sh)

The SoP.md entry notes: "aligned TASK lint enforcement in tools/lint/dp.sh". This refers to the validation logic that ensures TASK.md adheres to the schema.

- **Schema Enforcement:** The linter checks for the presence of specific headers like ## 1. Session State and ## 3.1 Freshness Gate.
- **Decimal Heading Scheme:** The linter enforces a strict decimal heading structure (e.g., 3.1, 3.2, 3.4.1). This prevents agents from inventing new section hierarchies that might confuse downstream parsers or human operators.
- **Drift Detection:** The linter likely checks for "forbidden tokens" or legacy patterns (like "Mandatory artifacts list") and fails the build if they are present.

The fact that the repo-gates workflow [1] passed for DP-OPS-0051 indicates that the new TASK.md structure satisfies the strict requirements of tools/lint/dp.sh.

4.3 Context Hygiene (tools/lint/context.sh)

The "Context Hazard" check is a critical security gate. PoT.md forbids the inclusion of docs/library/ in the global context manifest (ops/lib/manifests/CONTEXT.md).

- **Logic:** The script parses CONTEXT.md and greps for forbidden paths (agents/, tasks/, skills/).
- **Verification:** The SoP.md verification receipt confirms bash tools/lint/context.sh was run and passed. This proves that the remediation did not accidentally re-introduce the full library into the global context, maintaining the JIT (Just-In-Time) loading architecture.

4.4 Session Capture (ops/bin/open & ops/bin/dump)

These binaries form the "Bridge" between the disk state and the agent's context.

- **ops/bin/open:** Generates the OPEN artifact.[1] It injects the "Freshness Gate" data (Branch, HEAD, Porcelain) directly into the prompt. The audit of snippet [1] shows the prompt explicitly pointing to TASK.md as the "PRIMARY WORK SURFACE" but noting it "does NOT auto-load." This confirms the pointer-first logic: the prompt tells the agent *where* the work is, but forces the agent to read the file to get the details.
- **ops/bin/dump:** Generates the file content dump. The `` [1] shows that it respects the

.gitignore and Filing Doctrine, excluding storage/ (except handoffs/dumps) and projects/ (unless scoped).

5. The "Shadow Manifest" Anomaly and Residual Drift

Despite the success of the primary remediation, the forensic audit has uncovered a significant, high-severity governance gap: **The Shadow Manifest Anomaly**.

5.1 The llms.txt Canon Drift

PoT.md Section 1.3 (Canon Surfaces) explicitly lists llms.txt as the *only* canon surface for discovery.[1] It is the "discovery entry point."

However, the repository index [1] reveals a cluster of related files:

- llms.txt (The authorized surface)
- llms-full.txt (Unauthorized)
- llms-governance.txt (Unauthorized)
- llms-ops.txt (Unauthorized)
- llms-small.txt (Unauthorized)

The Violation: PoT.md Axiom 1.2 states: "duplication of canon outside SSOT is a failure state".[1] These additional files likely contain duplicates of the content in PoT.md, SoP.md, etc., tailored for different context window sizes. While technically useful, their existence is **unconstitutional** because PoT.md does not authorize them. They are "Ghost Canon" surfaces—files that act as authorities but have no legal standing.

5.2 The Shadow Manifest (ops/lib/manifests/LLMS.md)

The generation of these unauthorized bundles is controlled by ops/lib/manifests/LLMS.md, a file present in the index [1] and referencing in the llms binary spec.[1]

- **The Conflict:** PoT.md Section 1.3 lists ops/lib/manifests/CONTEXT.md as a canon surface. It *does not* list ops/lib/manifests/LLMS.md.
- **Implication:** A critical part of the system's "Run" logic (the generation of context bundles) is governed by a file that the constitution does not recognize. This is a "Shadow Manifest." If an agent were to consult PoT.md to find the list of valid manifests, it would miss LLMS.md.
- **Risk:** This creates a vector for "Deep Drift." An operator could modify LLMS.md to include restricted files (like secrets or deprecated agents), and because PoT.md doesn't recognize the file, standard governance checks might overlook it.

5.3 Technical Severity

The severity is classified as **Medium-High**.

- **Operational Impact:** Low. The system functions. The binaries read the shadow manifest and generate the bundles.
- **Governance Impact:** High. The repository is in a state of "Legal Incoherence." The PoT says one thing, but the file system does another. In a strict "Code is Law" environment, this is a failure state.

6. State of Play (SoP.md) Ledger Integrity

The integrity of the SoP.md ledger is the ultimate proof of the repository's history. The audit confirms that DP-OPS-0051 successfully restored continuity.

6.1 Timestamp and Sequence Analysis

Entry ID	Timestamp (UTC)	Status	Analysis
DP-OPS-0051	2026-02-12 00:07:59	**SUCCESS**	Remediation complete. Verification passed. New architecture deployed.
DP-OPS-0050	2026-02-12 00:02:54	**PARTIAL**	"Closeout Ledger Repair." Explicit acknowledgement of scrub failure.
DP-OPS-0047	2026-02-11 05:10:37	**SUCCESS**	Previous stable state.

The timeline reveals that the repair of 0050 and the execution of 0051 happened in rapid succession (within 5 minutes). This suggests a focused "Cleanup Session" where the operator explicitly addressed the debt of the previous run before moving forward.

6.2 The "Short Entry" Standard

The entries adhere to the policing rules defined in .github/workflows/sop_policing.yml.[1]

- **Constraint:** "Add a short SoP entry (what changed + why)."
- **Compliance:** The entries use bullet points for "Objective," "Verification," and "Functional receipts." They are concise, factual, and devoid of narrative fluff. This adherence ensures the ledger remains parseable and does not bloat the context window.

6.3 Resolution of the Scrub Paradox

The presence of the DP-OPS-0050 entry labeled "Closeout Ledger Repair" is the definitive proof that the Scrub Paradox is resolved. The system now has a protocol for handling failure: **Document the failure in the ledger.** By recording the "Dirty Tree" state as a historical fact, the system avoids the need to "fix" history, preserving the integrity of the chain.

7. Strategic Implications and Future Trajectory

The successful remediation of DP-OPS-0051 marks a maturity point for the Stela repository. It has moved from a "Text-Based" governance model to a "Tool-Enforced" model.

7.1 System Equilibrium

The repository is currently in a state of **High Equilibrium**.

- TASK.md is clean (template state).
- SoP.md is accurate and continuous.
- PoT.md rules (mostly) match the repository structure.
- The "Dirty Tree" exemption for TASK.md [1] allows for active work without triggering drift alarms.

7.2 The "Living Surface" vs. "Dead Ledger"

The distinction between TASK.md (Living) and SoP.md (Dead) is now architecturally enforced. TASK.md cannot become a permanent record because ops/bin/prune destroys it. SoP.md cannot become a workspace because it is append-only. This "separation of powers" is the strongest defense against entropy.

8. Conclusion and Strategic Recommendations

The forensic audit concludes that **DP-OPS-0051 was a comprehensive success**. The immediate threats of Context Hazard and Ghost Canon in TASK.md have been neutralized. The toolchain (prune, open, lint) effectively enforces the new "Pointer-First" doctrine.

However, the "Shadow Manifest" anomaly represents a latent risk that prevents total constitutional compliance.

8.1 Final Verdict

- **Remediation Status: COMPLETE**.
- **Ledger Integrity: RESTORED**.
- **Governance Compliance: PARTIAL** (due to LLMS Bundle Drift).

8.2 Recommendations for Next Dispatch Packet (DP-OPS-0052)

To achieve 100% compliance with the Policy of Truth, the next Dispatch Packet must address the "Shadow Manifest."

1. **Canonize LLMS.md:** Amend PoT.md Section 1.3 to explicitly list ops/lib/manifests/LLMS.md as a Canon Surface. This legitimizes the generation of context bundles.
2. **Authorize Bundle Artifacts:** Update PoT.md to explicitly authorize the existence of llms-full.txt, llms-ops.txt, etc., as "Generated Canon" or "Derived Artifacts." This resolves the technical Drift violation where files exist that are not permitted.
3. **Audit context.sh:** Ensure the tools/lint/context.sh script scans LLMS.md for Context Hazards (inclusion of library/ files), just as it scans CONTEXT.md. This closes the security loop on the shadow manifest.

With these changes, the Stela repository will achieve "Perfect Equilibrium," where every file on disk is accounted for, authorized, and governed by the Policy of Truth.

Works cited

1. dump-full-main-088b6100.txt